

Lab Assignment 13.3 – Code Refactoring Using AI Assistance

Name: Manupati Eshwar

Roll No: 2303A51495

Task 1: Refactoring – Removing Code Duplication

Code:

```
def calculate_rectangle(length, width):  
  
    area = length * width  
    perimeter = 2 * (length + width)  
    return area, perimeter  
  
# Rectangle 1  
area, perimeter = calculate_rectangle(5, 10)  
print("Area of Rectangle:", area)  
print("Perimeter of Rectangle:", perimeter)  
  
# Rectangle 2  
area, perimeter = calculate_rectangle(7, 12)  
print("Area of Rectangle:", area)  
print("Perimeter of Rectangle:", perimeter)  
  
# Rectangle 3  
area, perimeter = calculate_rectangle(10, 15)  
print("Area of Rectangle:", area)  
print("Perimeter of Rectangle:", perimeter)
```

Output:

```
Area of Rectangle: 50  
Perimeter of Rectangle: 30  
Area of Rectangle: 84  
Perimeter of Rectangle: 38  
Area of Rectangle: 150  
Perimeter of Rectangle: 50
```

```
main.py
1- def calculate_rectangle(length, width):
2-     """
3-     Calculate and return the area and perimeter of a rectangle.
4-
5-     Parameters:
6-     length (int/float): Length of the rectangle
7-     width (int/float): Width of the rectangle
8-
9-     Returns:
10-    tuple: (area, perimeter)
11-    """
12-    area = length * width
13-    perimeter = 2 * (length + width)
14-    return area, perimeter
15-
16-
17- # Rectangle 1
18- area, perimeter = calculate_rectangle(5, 10)
19- print("Area of Rectangle:", area)
20- print("Perimeter of Rectangle:", perimeter)
21-
22- # Rectangle 2
23- area, perimeter = calculate_rectangle(7, 12)
24- print("Area of Rectangle:", area)
25- print("Perimeter of Rectangle:", perimeter)
26-
27- # Rectangle 3
28- area, perimeter = calculate_rectangle(10, 15)
29- print("Area of Rectangle:", area)
30- print("Perimeter of Rectangle:", perimeter)
```

Output

```
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50

=== Code Execution Successful ===
```

Task 2: Refactoring – Optimizing Loops and Conditionals

Code:

```
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
```

```
# Convert list to set for faster lookup
names_set = set(names)
```

```
for name in search_names:
    if name in names_set:
        print(f"{name} is in the list")
    else:
        print(f"{name} is not in the list")
```

Output:

```
Charlie is in the list
Eve is not in the list
Bob is in the list
```

Performance Improvement:

Legacy Code Time Complexity: $O(n*m)$ due to nested loops.

Refactored Code Time Complexity: $O(n + m)$ using set lookup.

main.py	Output
<pre>1 names = ["Alice", "Bob", "Charlie", "David"] 2 search_names = ["Charlie", "Eve", "Bob"] 3 4 # Convert list to set for faster lookup 5 names_set = set(names) 6 7 for name in search_names: 8 if name in names_set: 9 print(f"{name} is in the list") 10 else: 11 print(f"{name} is not in the list")</pre>	<pre>Charlie is in the list Eve is not in the list Bob is in the list === Code Execution Success</pre>

Task 3: Refactoring – Extracting Reusable Functions

Code:

```
def calculate_total(price):
```

```
    tax_rate = 0.18
    tax = price * tax_rate
    return price + tax
```

Example 1

```
print("Total Price:", calculate_total(250))
```

Example 2

```
print("Total Price:", calculate_total(500))
```

Output:

Total Price: 295.0

Total Price: 590.0

main.py	Output
<pre>1 def calculate_total(price): 2 """ 3 Calculate total price including 18% tax. 4 5 Parameters: 6 price (int/float): Original price 7 8 Returns: 9 float: Total price including tax 10 """ 11 TAX_RATE = 0.18 12 tax = price * TAX_RATE 13 return price + tax 14 15 16 print("Total Price:", calculate_total(250)) 17 print("Total Price:", calculate_total(500))</pre>	<pre>Total Price: 295.0 Total Price: 590.0 === Code Execution Successful ===</pre>

Task 4: Refactoring – Replacing Hardcoded Values with Constants

Code:

```
PI = 3.14159
```

```
RADIUS = 7
```

```
area = PI * (RADIUS ** 2)
```

```
circumference = 2 * PI * RADIUS
```

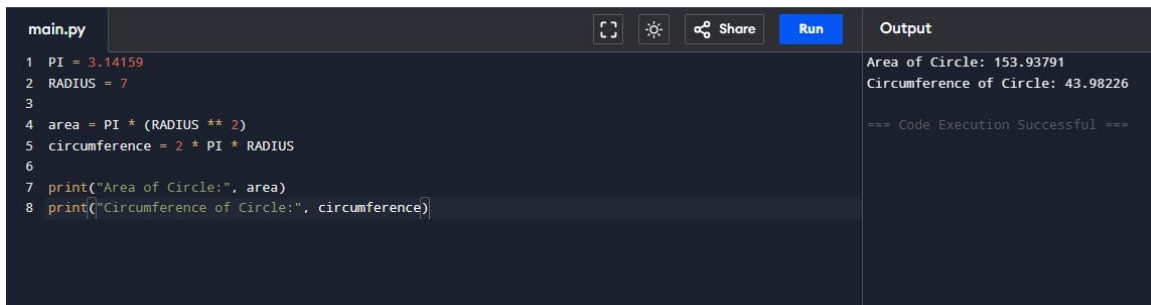
```
print("Area of Circle:", area)
```

```
print("Circumference of Circle:", circumference)
```

Output:

```
Area of Circle: 153.93891
```

```
Circumference of Circle: 43.98226
```



```
main.py  [Icons]  Share  Run  Output
1  PI = 3.14159
2  RADIUS = 7
3
4  area = PI * (RADIUS ** 2)
5  circumference = 2 * PI * RADIUS
6
7  print("Area of Circle:", area)
8  print("Circumference of Circle:", circumference)

Area of Circle: 153.93791
Circumference of Circle: 43.98226

=== Code Execution Successful ===
```

Task 5: Refactoring – Improving Variable Naming and Readability

Code:

```
# Base of the triangle
```

```
base = 10
```

```
# Height of the triangle
```

```
height = 20
```

```
# Calculate area of triangle using formula: (base * height) / 2
```

```
area_of_triangle = (base * height) / 2
```

```
print(area_of_triangle)
```

Output:

```
100.0
```

main.py				 Share	Run	Output
<pre>1 # Base of the triangle 2 base = 10 3 4 # Height of the triangle 5 height = 20 6 7 # Formula: Area = (base * height) / 2 8 area_of_triangle = (base * height) / 2 9 10 print(area_of_triangle)</pre>						100.0 === Code Execut: